Der Befehlssatz der Registermaschine Johnny umfasst die folgenden Befehle:

```
TAKE x : kopiert den Wert aus Speicherzelle x in den Akkumulator
ADD x : Akkumulator um Inhalt des Registers x erhöhen
SUB x : Akkumulator um Inhalt des Registers x erniedrigen
SAVE x : Inhalt des Akkumulators im Register x speichern
JMP x : springe zu Register x
TST x : fals Register x = 0, springe ins übernächste Register
INC x : Erhöhe Inhalt des Registers x um 1
DEC x : Erniedrige Inhalt des Registers x um 1
NULL x : Setze Inhalt des Registers x auf 0
HLT x : Das Programm hält an.
```

Schreibweise: Wir schreiben [10] wenn wir die Zahl an der Speicherstelle (=Register) 10 meinen.
Johnny arbeitet nur mit nicht-negativen Zahlen: Ist ein Ergebnis kleiner als 0, steht dort 0.

1. (5 Punkte) a) Nenne die Komponenten eines Von-Neumann-Rechners und beschreibe jeweils kurz ihre Aufgabe.
   b) Erkläre den Unterschied zwischen dem Adressbus und dem Datenbus.

Lösung:

a)
- **Rechenwerk** (ALU): führt Rechenoperationen und logische Verknüpfungen aus.
- **Steuerwerk**: holt die Befehle nacheinander aus dem Speicher, entschlüsselt sie und steuert die anderen Komponenten so, dass der Befehl ausgeführt wird.
- **Speicherwerk** (Arbeitsspeicher): speichert Programme und Daten.
- **Ein-/Ausgabewerk**: verbindet den Rechner mit Ein- und Ausgabegeräten.
- **Bussystem**: verbindet die Komponenten (Adressbus, Datenbus, Steuerbus).
- **Register**: schnelle Speicher im Prozessor, z. B. Akkumulator, Befehlsregister, Befehlszähler.

Rechenwerk und Steuerwerk bilden zusammen den Prozessor (CPU).

b) Über den **Adressbus** wird die Adresse übertragen, also welche Speicherzelle gelesen oder beschrieben wird. Über den **Datenbus** wird der Inhalt übertragen, also die Daten oder Befehle selbst.

2. (3 Punkte) In einer Speicherzelle von Johnny steht der Wert `02.015`. Erläutere, ob es sich dabei um eine Zahl oder um einen Befehl handelt. Gehe dabei auf das Von-Neumann-Prinzip ein, nach dem Programm und Daten im selben Speicher liegen.

Lösung: Das lässt sich am Wert allein nicht entscheiden. Beim Von-Neumann-Rechner liegen Programm und Daten im selben Speicher, und einer Speicherzelle sieht man nicht an, was sie enthält.

Wird die Zelle über den Befehlszähler geholt und ins Befehlsregister übernommen, wird sie als Befehl `ADD 15` ausgeführt. Wird sie dagegen von einem Befehl wie `TAKE` gelesen, wird sie als Zahl (2015) verwendet.

3. (4 Punkte) Ein Von-Neumann-Rechner arbeitet in einem ständigen Befehlszyklus.

   a) Beschreibe die drei Phasen des Befehlszyklus.

   b) Gib die Mikrobefehle an, mit denen Johnny in der ersten Phase einen Befehl holt, und erkläre jeweils, was passiert.

Lösung:

a)
- **Holen** (Fetch): Der Befehl, auf den der Befehlszähler zeigt, wird aus dem Speicher ins Befehlsregister geholt.
- **Entschlüsseln** (Decode): Das Steuerwerk stellt fest, um welchen Befehl es sich handelt.
- **Ausführen** (Execute): Der Befehl wird ausgeführt. Der Befehlszähler wird auf den nächsten Befehl gesetzt (bei einem Sprung auf die Sprungadresse). Danach beginnt der Zyklus von vorn.

b)
```
pc->ab    Adresse des nächsten Befehls auf den Adressbus
ram->db   Befehl aus dem RAM auf den Datenbus
db->ins   Befehl ins Befehlsregister
ins->mc   Sprung zum Mikroprogramm des Befehls (Entschlüsseln)
```

4. (4 Punkte) Schreibe ein Programm für folgenden Ablauf:

```
[14] = [12] + [13]
[12] = [12] - 1
[13] = [13] + 1
[15] = [12] + [13]
```

Lösung:

```
1  TAKE 12
2  ADD 13
3  SAVE 14

4  DEC 12
5  INC 13

6  TAKE 12
7  ADD 13
8  SAVE 15

9  HLT 0
```

5. (4 Punkte) Schreibe ein Programm für folgenden Ablauf:

```
[12] = [10] * 4
```

Lösung:

```
1  TAKE 10
2  ADD 10
3  ADD 10
4  ADD 10
5  SAVE 12
6  HLT 0
```

6. (4 Punkte) Schreibe ein Programm für folgenden Ablauf:

```
[14] = [13] * [12]
```

Lösung:

```
0 NULL 14
1 TST 12
2 JMP 4
3 HLT 0
4 TAKE 14
5 ADD 13
6 SAVE 14
7 DEC 12
8 JMP 1
```

7. (4 Punkte) Gegeben:

```
[17] teilbar durch [18]
```

Schreibe ein entsprechendes Programm:

```
[19] = [17] / [18]
```

Lösung:

```
0 NULL 19
1 TST 17
2 JMP 4
3 HLT 0
4 TAKE 17
5 SUB 18
6 SAVE 17
7 INC 19
8 JMP 1
```

8. (4 Punkte) Schreibe ein entsprechendes Programm:

```
[25] = 1 + 2 + ... + [24]
```

Lösung:

```
0 NULL 25
1 TST 24
2 JMP 4
3 HLT 0

4 TAKE 25
5 ADD 24
6 SAVE 25
7 DEC 24
8 JMP 1
```

9. (4 Punkte) Schreibe ein entsprechendes Programm:

```
[30] und [31] werden vertauscht
```

Lösung:

```
0 TAKE 30
1 SAVE 32

2 TAKE 31
3 SAVE 30

4 TAKE 32
5 SAVE 31

6 HLT 0
```

Die Mikrobefehle der Registermaschine Johnny sind:

```
db->ram  : Inhalt des Datenbus im RAM speichern
ram->db  : Inhalt des RAM auf den Datenbus legen
db->ins  : Inhalt des Datenbus ins Befehlsregister übernehmen
db->acc  : Inhalt des Datenbus in den Akkumulator übernehmen
acc->db  : Inhalt des Akkumulators auf den Datenbus legen
plus     : Akkumulator um Inhalt des Datenbus erhöhen
minus    : Akkumulator um Inhalt des Datenbus erniedrigen
acc++    : Akkumulator um 1 erhöhen
acc--    : Akkumulator um 1 erniedrigen
acc=0    : Akkumulator auf 0 setzen
pc->ab   : Befehlszähler auf den Adressbus legen
pc++     : Befehlszähler um 1 erhöhen
=0:pc++  : falls Akkumulator = 0, Befehlszähler um 1 erhöhen
ins->ab  : Adressteil des Befehlsregisters auf den Adressbus legen
ins->pc  : Adressteil des Befehlsregisters in den Befehlszähler schreiben
ins->mc  : zum Mikroprogramm des Befehls im Befehlsregister springen
mc=0     : zurück zum Anfang des Mikrocodes (FETCH)
stop     : Programm beenden
```

10. (2 Punkte) Schreibe mit den Mikrobefehlen von Johnny das Mikroprogramm für den Assemblerbefehl `ADD x`. Erinnerung: `ADD x` erhöht den Akkumulator um den Inhalt von Register x.

Lösung:

```
ADD x:
  ins->ab
  ram->db
  plus
  pc++
  mc=0
```

11. (4 Punkte) Der Befehlssatz von Johnny soll um einen neuen Befehl `DBL x` erweitert werden, der den Inhalt von Register x verdoppelt:

```
DBL 10   bedeutet   [10] = 2 * [10]
```

Schreibe das Mikroprogramm für `DBL x` und erläutere die einzelnen Schritte.

Lösung:

```
ins->ab     Adresse x auf den Adressbus
ram->db     [x] auf den Datenbus
db->acc     ACC = [x]
plus        ACC = 2 * [x]   ([x] steht noch auf dem Datenbus)
acc->db     Ergebnis auf den Datenbus
db->ram     [x] = 2 * [x]   (x steht noch auf dem Adressbus)
pc++        Befehlszähler auf den nächsten Befehl
mc=0        zurück zum Holen
```